

Group ID 和 Chat ID 分配规则

概述

系统中有两类 ID： **群组 ID**（`user_groups` 表的 `group_id`）和 **聊天室 ID**（`groups_chat` 表的 `group_id`）。它们采用不同的分配策略：系统群组使用基于 UID 的可预测 ID，自定义群组和聊天室使用加密安全的随机 ID。

一、Group ID（用户群组容器）

对应表: `user_groups.group_id`

1. 系统群组 — `ensure_default_groups()`

文件: `app.py:1036-1075`

用户注册或登录时自动创建，ID 规则为 **固定前缀 + 用户 UID**。

即时聊天群组

```
# app.py:1055
group_id = 'sys_im_' + uid
```

组件	值	说明
前缀	<code>sys_im_</code>	固定字符串，表示"系统-即时聊天"
变量部分	<code>uid</code>	当前用户的学号/工号
示例	<code>sys_im_2022010101</code>	学号为 2022010101 的用户的即时聊天群组

特点:

- 每个用户**唯一**一个即时聊天群组
- ID 可预测，通过 UID 可直接推导
- 创建时机：首次注册或首次登录时（`register()` 和 `login()` 中都调用了 `ensure_default_groups(uid)`）
- 创建后会检查用户是否已有两个系统群组（`existing >= 2`），避免重复创建

私聊群组

```
# app.py:1067
group_id = 'sys_private_' + uid
```

组件	值	说明
前缀	<code>sys_private_</code>	固定字符串，表示"系统-私聊"
变量部分	<code>uid</code>	当前用户的学号/工号
示例	<code>sys_private_2022010101</code>	学号为 2022010101 的用户的私聊群组

特点:

- 每个用户**唯一**一个私聊群组
- ID 可预测，通过 UID 可直接推导
- 与即时聊天群组在同一函数 `ensure_default_groups()` 中创建

2. 自定义群组（用户创建）

文件: `app.py:692-734` — `create_user_group()`

```
# app.py:702
group_id = secrets.token_hex(8)
```

项目	说明
生成函数	<code>secrets.token_hex(8)</code>
输出长度	16 个字符 (8 字节 → 16 个十六进制字符)
字符集	<code>0-9a-f</code>
随机性	密码学安全 (<code>secrets</code> 模块, 适用于安全场景)
示例	<code>a1b2c3d4e5f6g7h8</code>

特点:

- **不可预测**, 无法从 ID 反推创建者
- 碰撞概率极低 ($16^{16} \approx 1.8 \times 10^{19}$ 种可能)
- 每次用户点击"新建群组"时生成

二、Chat ID (聊天室/群聊房间)

对应表:`groups_chat.group_id` (表字段名虽叫 `group_id`, 但实际是聊天室的 ID)

1. 创建群聊 (直接创建聊天室)

文件:`app.py:1094-1120` — `create_group()`

```
# app.py:1102
group_id = secrets.token_hex(8)
```

项目	说明
生成函数	<code>secrets.token_hex(8)</code>
输出长度	16 个字符
字符集	<code>0-9a-f</code>
随机性	密码学安全
示例	<code>d4e5f6a7b8c9d0e1</code>

2. 在群组内创建聊天室

文件:`app.py:895-927` — `create_chat_in_group()`

```
# app.py:911
chat_id = secrets.token_hex(8)
```

项目	说明
生成函数	<code>secrets.token_hex(8)</code>
输出长度	16 个字符
字符集	<code>0-9a-f</code>
随机性	密码学安全
示例	<code>f1e2d3c4b5a69788</code>

说明: 虽然变量名是 `chat_id`，但它是写入 `groups_chat` 表的 `group_id` 字段，与直接创建群聊使用的是同一个 ID 空间。

三、ID 分配总结表

类型	范围	生成方式	格式	示例	代码位置
系统-即时聊天群组	<code>user_groups.group_id</code>	<code>'sys_im_' + uid</code>	<code>sys_im_{uid}</code>	<code>sys_im_2022010101</code>	<code>app.py:1055</code>
系统-私聊群组	<code>user_groups.group_id</code>	<code>'sys_private_' + uid</code>	<code>sys_private_{uid}</code>	<code>sys_private_2022010101</code>	<code>app.py:1067</code>
自定义群组	<code>user_groups.group_id</code>	<code>secrets.token_hex(8)</code>	16 位十六进制	<code>a1b2c3d4e5f6g7h8</code>	<code>app.py:702</code>
聊天室 (直接创建)	<code>groups_chat.group_id</code>	<code>secrets.token_hex(8)</code>	16 位十六进制	<code>d4e5f6a7b8c9d0e1</code>	<code>app.py:1102</code>
聊天室 (群组内创建)	<code>groups_chat.group_id</code>	<code>secrets.token_hex(8)</code>	16 位十六进制	<code>f1e2d3c4b5a69788</code>	<code>app.py:911</code>

四、ID 的使用方式

在 URL 和 API 中

所有 ID 都以路径参数形式传递：

```
GET /api/user-groups/{group_id}           ← 用户群组 ID
GET /api/user-groups/{group_id}/detail    ← 用户群组 ID
GET /api/groups/{chat_id}/messages        ← 聊天室 ID
POST /api/groups/{chat_id}/send           ← 聊天室 ID
```

在前端存储

ID 被存储在 DOM 元素的 `data-` 属性中，供事件处理使用：

```
// groups.html:347 - 群组列表
<div class="group-item" data-gid="{g.group_id}" ...>

// groups.html:414 - 聊天室列表
<div class="chat-item" data-chatid="{c.chat_id}" ...>

// groups.html:673 - 添加聊天室对话框
<input type="checkbox" data-chatid="{g.group_id}" ...>
```

系统群组的特殊处理

系统群组 (`group_type = 'system'`) 在前端有特殊标识：

```
// groups.html:352
${g.group_type === 'system' ? '<span class="system-badge">系统</span>' : ''}
```

五、`secrets.token_hex(8)` 详解

```
import secrets
secrets.token_hex(8) # → 'a1b2c3d4e5f6g7h8'
```

方面	说明
模块	Python 标准库 <code>secrets</code> — 专用于密码学安全的随机数
参数 <code>8</code>	生成 8 个随机字节
输出	16 个十六进制字符（每个字节编码为 2 个 hex 字符）
碰撞概率	$1 / 2^{64} \approx 5.4 \times 10^{-20}$ ，可忽略不计
对比 <code>random</code>	<code>secrets</code> 比 <code>random</code> 更安全，适合生成 ID、Token、密钥等

六、注意事项

- 1. **变量名混淆:** `groups_chat` 表的主键字段名为 `group_id`，但实际代表的是"聊天室 ID"。代码中有时用 `group_id` 有时用 `chat_id` 指代同一事物，阅读时需注意上下文。
- 2. **系统群组 ID 可预测:** `sys_im_{uid}` 和 `sys_private_{uid}` 可以通过 UID 推算出来。这不是安全问题，因为这些群组是通过 `INSERT OR IGNORE` 写入的，且用户只能访问自己有权限的群组。
- 3. **ID 不可修改:** 所有 `group_id` 和 `chat_id` 在创建后硬编码在关联表中，不支持修改。
- 4. **系统群组仅自动创建一次:** `ensure_default_groups()` 中通过 `existing >= 2` 检查确保不会重复创建。
- 5. **关联表使用同一 ID:** `user_group_chats` 中的 `chat_id` 字段引用的就是 `groups_chat.group_id`，两个 ID 空间完全重叠。